a. Identify/discover a use case

Solve Sudoku Puzzles.

b. Provide a brief description for the use case

This use case allows a user to run the Sudoku solver program to automatically load five partially completed Sudoku puzzle files, find all valid solutions using a depth-first search with backtracking, and display the results.

c. Provide a step-by-step flow (outline) of the events for the use case

1. User executes the Sudoku solver program.
2. System loads the target puzzle file (e.g., puzzle1.txt).
3. System validates the consistency of the initial puzzle clues.
4. System executes the backtracking search algorithm to find solutions.
5. System prints the target file name and original puzzle layout.
6. System displays all valid solutions or a "No solution found" message.
7. System repeats steps 2-6 for the remaining puzzle files.

d. Provide a detailed description for the use case

- **User executes the Sudoku solver program:** The user runs the compiled Assignment4.exe executable file from the command line.
- **System loads the target puzzle file:** The system iterates through an array of five specified text files (puzzle1.txt through puzzle5.txt). It reads the 9x9 grid data, converting numerical string tokens into integers and underscore characters ("_") into 0s to represent blank cells. If a file cannot be opened, is missing entries, or contains invalid tokens, the system logs an error message and halts processing for that specific file.
- **System validates the consistency of the initial clues:** Before solving, the system temporarily removes each provided clue and verifies that placing it back does not violate standard Sudoku rules (no duplicate numbers in the same row, column, or 3x3 block). If the starting clues contradict each other, the system flags the puzzle to prevent an infinite or failed search.
- **System executes the backtracking search algorithm:** The system finds the next empty cell and sequentially attempts to insert candidate numbers from 1 to 9. If a candidate number is legally allowed, the system temporarily places it and recursively calls itself to evaluate the next cell. If a path fails to yield a solution, the system backtracks by resetting the cell to 0 and trying the next candidate. When the system successfully fills all 81 cells, it records the completed grid as a valid solution and backtracks immediately to search for additional valid configurations.
- **System prints the target file name and original puzzle layout:** The system outputs the name of the processed file and displays the 9x9 initial puzzle grid to the console, using underscores to represent the blank cells.

- **System displays all valid solutions:** If solutions were found, the system prints each sequentially numbered solution grid and the total count of discovered solutions. If the search algorithm exhaustively checks all possibilities without finding a valid configuration, the system prints "No solution found".

